

Análise de Algoritmos de Ordenação em Listas Estáticas e Dinâmicas para um Ranking de Jogos

1st Ana Aisha Tomaz de Moraes 2nd Espedito Ramom Mascena Ricarto 3rd Pedro Kauan Cardoso da Silva
Engenharia de Software *Engenharia de Software* *Engenharia de Software*
Universidade Federal do Cariri (UFCA) *Universidade Federal do Cariri (UFCA)* *Universidade Federal do Cariri (UFCA)*
Juazeiro do Norte, Brasil Juazeiro do Norte, Brasil Juazeiro do Norte, Brasil

4th Ramona Vitória Clemente Cardoso 5th Sabrina Alencar Soares 6th Sebastião Sousa Soares
Engenharia de Software *Engenharia de Software* *Engenharia de Software*
Universidade Federal do Cariri (UFCA) *Universidade Federal do Cariri (UFCA)* *Universidade Federal do Cariri (UFCA)*
Juazeiro do Norte, Brasil Juazeiro do Norte, Brasil Juazeiro do Norte, Brasil

Resumo—Este artigo apresenta uma análise empírica do desempenho de algoritmos clássicos de ordenação (Bubble Sort, Selection Sort, Insertion Sort, Quick Sort e Merge Sort) aplicados a duas estruturas de dados distintas: listas estáticas (vetores) e listas encadeadas dinâmicas. O estudo foi contextualizado na implementação de um sistema de ranking de jogadores. Foram realizados experimentos com diferentes volumes de dados e diferentes estados iniciais de ordenação. Os resultados indicam que algoritmos com complexidade $O(N^2)$ tornam-se rapidamente ineficientes à medida que o tamanho da entrada cresce, enquanto algoritmos $O(N \log N)$ apresentam desempenho mais estável. Entre os algoritmos analisados, o Merge Sort apresentou o comportamento mais consistente nos diferentes cenários avaliados.

Index Terms—Algoritmos de Ordenação, Estruturas de Dados, Complexidade de Tempo, Engenharia de Software.

I. INTRODUÇÃO

Sistemas de ranqueamento e *leaderboards* digitais são amplamente utilizados em plataformas de jogos e aplicações online. Nesses sistemas, a ordenação frequente de dados é uma operação essencial, pois a posição de cada jogador precisa ser atualizada conforme novos resultados são registrados.

A eficiência dessas operações depende diretamente da combinação entre a estrutura de dados utilizada e o algoritmo de ordenação escolhido. Estruturas diferentes podem influenciar significativamente o desempenho de um mesmo algoritmo, principalmente quando o volume de dados cresce.

Diante disso, este trabalho tem como objetivo analisar empiricamente o desempenho de diferentes algoritmos de ordenação quando aplicados a duas estruturas de dados distintas: listas estáticas (vetores) e listas encadeadas dinâmicas. A partir dos experimentos realizados, busca-se observar como os resultados práticos se relacionam com as análises teóricas de complexidade de tempo.

II. METODOLOGIA DE TESTE

Para realizar a análise empírica, foi desenvolvido um gerador de dados capaz de produzir conjuntos de diferentes tamanhos. Os experimentos foram conduzidos com três volumes de

entrada: 100 elementos (pequeno), 1.000 elementos (médio) e 10.000 elementos (grande).

Além disso, três configurações iniciais dos dados foram consideradas: elementos em ordem aleatória, elementos previamente ordenados e elementos em ordem inversa. Esses cenários permitem observar o comportamento dos algoritmos em diferentes condições de entrada.

A medição do tempo de execução foi realizada utilizando funções de temporização de alta resolução. Em sistemas POSIX foi utilizada a função `clock_gettime()` com o relógio `CLOCK_MONOTONIC`, enquanto em ambientes Windows foi utilizada a API `QueryPerformanceCounter()`. Apenas o trecho correspondente ao processo de ordenação foi considerado na medição, de forma a evitar que operações de leitura de arquivos interferissem nos resultados.

Cada experimento foi repetido 100 vezes, sendo utilizada a média aritmética dos tempos obtidos para representar o resultado final.

Ambiente de execução: os testes foram realizados em um notebook Dell Inspiron i15-3501-A25P, equipado com processador Intel Core i3 e 4 GB de memória RAM, executando o sistema operacional Windows. O código foi compilado utilizando o compilador GCC com a flag `-O0`, evitando otimizações automáticas do compilador que poderiam alterar o comportamento observado dos algoritmos.

O código-fonte completo utilizado nos experimentos, bem como os arquivos de entrada empregados nos testes, está disponível em um repositório público para fins de reprodutibilidade dos resultados [3].

III. ALGORITMO SELECIONADO

Com base na análise teórica e nos resultados experimentais obtidos, o algoritmo selecionado como mais adequado para o problema proposto foi o *Merge Sort*. Esse algoritmo segue o paradigma de divisão e conquista, no qual o problema é dividido em partes menores que são resolvidas recursivamente e posteriormente combinadas.

O funcionamento do algoritmo consiste em dividir repetidamente o conjunto de dados em duas partes até que cada subdivisão contenha apenas um elemento. Em seguida, essas partes são combinadas novamente de forma ordenada por meio da operação de *merge*.

No contexto deste trabalho, o Merge Sort foi implementado de forma recursiva tanto para a lista estática quanto para a lista encadeada dinâmica. Na versão aplicada ao vetor foram utilizados vetores auxiliares temporários durante o processo de intercalação. Já na versão aplicada à lista encadeada, a ordenação foi realizada reorganizando os ponteiros dos nós da própria estrutura.

A. Pseudocódigo do Merge Sort

```

MergeSort(lista)
if lista possui mais de um elemento then
    dividir lista em duas metades
    MergeSort(metade esquerda)
    MergeSort(metade direita)
    combinar as duas metades ordenadas
end if

```

B. Complexidade do Algoritmo

A complexidade de tempo do Merge Sort permanece a mesma nos três cenários clássicos de análise:

- Melhor caso: $O(N \log N)$
- Caso médio: $O(N \log N)$
- Pior caso: $O(N \log N)$

Essa estabilidade ocorre porque o algoritmo sempre divide o problema em duas partes aproximadamente iguais, independentemente da ordem inicial dos dados.

IV. RESULTADOS EMPÍRICOS E DISCUSSÃO

A. Análise Geral de Desempenho

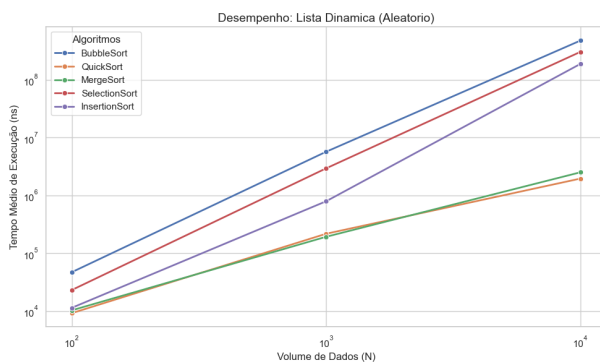


Figura 1. Tempo médio de execução dos algoritmos operando sobre lista encadeada dinâmica no cenário aleatório.

Os resultados obtidos confirmam o comportamento esperado a partir da análise teórica de complexidade. Algoritmos quadráticos ($O(N^2)$), como Bubble Sort e Selection Sort, apresentaram crescimento rápido no tempo de execução à medida que o tamanho da entrada aumentou.

Nos testes com 10.000 elementos, esses algoritmos atingiram tempos significativamente superiores aos demais, tornando-se pouco viáveis para cenários com grande volume de dados.

Já os algoritmos baseados em divisão e conquista apresentaram desempenho consideravelmente superior. O Quick Sort demonstrou excelente desempenho em entradas aleatórias, porém apresentou degradação em cenários desfavoráveis, como quando os dados estavam previamente ordenados ou inversos.

Por outro lado, o Merge Sort manteve desempenho relativamente estável em todos os cenários analisados. Nos testes realizados, seus tempos variaram entre aproximadamente 1,1 ms e 2,9 ms, independentemente da estrutura de dados ou da ordem inicial dos elementos.

B. Análise de Anomalias Práticas vs. Teóricas

Durante a análise dos resultados observou-se que alguns comportamentos práticos diferiram das previsões teóricas esperadas para determinados algoritmos. Essas variações podem estar associadas tanto à forma de implementação das estruturas de dados quanto a características do ambiente de execução, como organização da memória e comportamento do processador.

1) *O Efeito do Branch Prediction*: Um exemplo foi observado no Quick Sort aplicado à lista estática nos cenários de pior caso (dados ordenados e inversamente ordenados). Teoricamente, nesses cenários o algoritmo apresenta complexidade $O(N^2)$. Assim, ao aumentar o tamanho da entrada de 1.000 para 10.000 elementos (um fator de 10), seria esperado um crescimento próximo de 100 vezes no tempo de execução.

Nos experimentos realizados, entretanto, o tempo médio aumentou de 4,2 ms (conjunto médio) para 23,0 ms (conjunto grande), um crescimento bem menor do que o esperado pela análise teórica do algoritmo. Uma possível explicação para esse comportamento está relacionada ao mecanismo de *Branch Prediction* presente em processadores modernos. Quando os dados seguem padrões previsíveis, como em vetores já ordenados, o processador consegue antecipar o resultado de determinadas instruções condicionais, reduzindo penalidades de execução associadas a desvios de fluxo e melhorando o desempenho observado.

2) *O Comportamento LIFO da Lista Dinâmica*: Outro comportamento interessante foi identificado no Bubble Sort aplicado à lista encadeada dinâmica. Nesse caso, observou-se um desempenho aparentemente invertido entre os cenários Ordenado e Inverso: o algoritmo apresentou tempo muito baixo no cenário Inverso (0,10 ms) e tempo significativamente maior no cenário Ordenado (653 ms).

Esse comportamento está relacionado à forma como os elementos foram inseridos na lista encadeada. A inserção foi realizada sempre no início da estrutura (*head*), operação que possui custo $O(1)$, mas que resulta em uma organização semelhante ao comportamento de uma pilha (LIFO – *Last In, First Out*). Dessa forma, quando os dados eram importados de um arquivo CSV previamente ordenado, a própria construção

da lista acabava invertendo a ordem efetiva dos elementos na memória. Como consequência, o cenário conceitualmente “ordenado” passava a se comportar como um caso próximo do pior caso para o algoritmo, explicando os tempos observados nos experimentos.

V. CONCLUSÃO

A análise realizada neste trabalho permitiu observar, na prática, o impacto da escolha do algoritmo de ordenação e da estrutura de dados utilizada em um sistema de ranqueamento.

Os experimentos evidenciaram que algoritmos de complexidade quadrática tornam-se rapidamente inviáveis conforme o volume de dados aumenta. Em contraste, algoritmos com complexidade $O(N \log N)$ demonstraram comportamento mais estável e escalável.

Entre os algoritmos analisados, o Merge Sort apresentou o desempenho mais consistente nos diferentes cenários testados. Sua combinação com a estrutura de lista encadeada dinâmica mostrou-se adequada para aplicações de ranqueamento que exigem processamento frequente de grandes conjuntos de dados.

REFERÊNCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] N. Wirth, *Algorithms + Data Structures = Programs*. Englewood Cliffs, NJ: Prentice-Hall, 1976.
- [3] Repositório do projeto. Disponível em: <https://github.com/aishatomaz/jogadores-ranking>